Capstone — Low-rank frames vs differences

Launch Fri Dec 4 · Studio Mon Dec 7 · Gallery Wed Dec 9

Last updated: October 8, 2026.

Dates are on the schedule. There is **no quiz on SVD**. Friday Dec 4 is **project launch** (this packet) with the SVD lecture and Lab 6 (Fit and Compress). The graded product is this project (Canvas file upload).

Octave: Lachniet Ch. 5 (SVD) and Ch. 7 (image compression). Python: NumPy `svd` plus a simple image/array load. Syntax pointers: Resources.

The question

You get (or build) **three** grayscale frames F_1, F_2, F_3, each an $m \times n$ matrix of intensities.

Compare two encoding strategies with rank-k SVD approximations:

Plan	What you compress with a rank-k SVD
A — Independent frames	F_1, F_2, and F_3 separately
B — Frame + differences	F_1, then $D_2 = F_2 - F_1$, then $D_3 = F_3 - F_2$

Driving question: For your data, when (if ever) is Plan B better than Plan A on **accuracy**, **storage**, and **runtime / resource use** — including the cost of computing the SVDs?

Computing an SVD is **not** free. Low-rank storage or faster later multiplies only help when the matrix is well approximated at modest k **and** you reuse the representation enough (or care enough about storage/transmission) to justify the factorization cost. Image examples are instructional.

Timeline

When	What
Fri Dec 4	**Project launch** in Lecture 40 (after Quiz 11 / with Lab 6). Skim this packet; start choosing data.
Mon Dec 7	Studio — implement, measure, debug.
Wed Dec 9	Gallery — short presentation or station; **Canvas upload due** (end of day unless announced otherwise).

What to turn in (Canvas)

One PDF (or short report + clearly labeled figures), **individual** submission, about **1–2 pages of writing** plus figures/tables:

1. **Data.** Where the three frames came from (phone stills, course sample, synthetic frames, etc.) and size $m \times n$.
2. **Methods.** Rank(s) k you tried; how you formed D_2, D_3; how you rebuilt approximate frames under Plan B.
3. **Evidence (required numbers).**
   - Relative reconstruction error vs k for Plan A and Plan B (plot or table).
   - Storage count: full mn vs rank-k cost $\approx k(m + n + 1)$ per compressed matrix (and totals for A vs B).
   - Timing: at least one honest runtime comparison (e.g. SVD time vs applying a low-rank map, or Plan A vs B factorization time for the same k).
4. **Conclusion.** One clear recommendation: for *your* frames, prefer A, prefer B, or "depends," with a quantitative reason and **one limitation**.

Gallery: 2–3 minute lightning talk **or** a station with one figure people can read in 30 seconds. Peers may ask about your numbers.

Success criteria (how it is graded, 25 points)

Criterion	Points	What "good" looks like
Question & setup	5	Clear Plan A vs B; data and k choices stated
Correct SVD / rebuild	5	Rank-k approximation used correctly; Plan B rebuilds F_2, F_3 from F_1, D_2, D_3
Quantitative evidence	8	Error, storage, and at least one runtime/resource comparison with readable figure or table
Interpretation	5	Recommendation tied to numbers; names the payback / limitation of computing SVDs
Communication	2	Gallery contribution intelligible; write-up readable

Lab 6 check-in initials (Fit and Compress on launch day) are **effort credit** in the Labs group (separate from these 25 capstone points).

Technical notes

Storage (order of magnitude).

- One full frame: mn numbers.
- One rank-k factorization: about $k(m+n+1)$ numbers (columns of U_k, V_k, and k singular values).
- Plan A total (three frames): about $3k(m+n+1)$.
- Plan B total (three matrices): also about $3k(m+n+1)$ at the same k — so Plan B wins on *storage* only if you can use a **smaller** k for differences while matching Plan A's error, or if you choose different k per matrix on purpose.

Error. For a matrix A and approximation A_k, a standard relative Frobenius error is

$$\frac{\|A - A_k\|_F}{\|A\|_F}.$$

Under Plan B, measure error on the **reconstructed frames** $\widehat{F}_2 = \widehat{F}_1 + \widehat{D}_2$, etc., not only on the difference matrices.

Software sketch

Task	Octave	Python
SVD	`[U,S,V] = svd(A)`	`U, S, Vt = np.linalg.svd(A, full_matrices=False)`
Rank-k rebuild	first k columns / σ's	`U[:,:k] @ np.diag(S[:k]) @ Vt[:k,:]`
Frobenius norm	`norm(A,'fro')`	`np.linalg.norm(A, 'fro')`

Use grayscale (2D arrays). Color is optional enrichment only.

Practice packet (warm-up before studio)

About 14 prompts. Source key: Assessments.

- S 7.1 #1, #2, #3, #5, #6
- L Ch. 5 Ex. 7: small SVD
- L Ch. 5 Ex. 8: pseudoinverse/least-squares connection
- L Ch. 7 Project 7.1, Problems 1–3: image compression
- C-Cap.1: Derive full storage mn.
- C-Cap.2: Derive rank-k storage $k(m+n+1)$.
- C-Cap.3: Plot relative reconstruction error versus k.
- C-Cap.4: Compare full and factorized matrix-vector multiplication times and estimate when the initial SVD cost pays for itself.

Learning objectives

See Lecture 40 (launch), Lecture 41 (studio), and Lecture 42 (gallery).